

網站部署平台選擇 心得與實務指南

以提摩設計網站為例：Vercel、Zeabur、Railway 與其他方案的使用情境、優劣勢與遷移策略

核心問題

部署平台不是功能清單競賽。真正要選的是：現在願意承擔多少維運，以及未來希望網站長成單一官網、混合式系統，還是一套由多個服務組成的產品。

整理日期：2026 年 8 月 3 日

範圍：目前提摩 Next.js 網站、Supabase 後端，以及未來可能加入的表單、自動化、LINE、排程與背景服務

我的選擇心得

一開始我以為，Zeabur 因為能部署更多服務，可能是比 Vercel 更「一勞永逸」的起點。重新把平台定位、目前計費模式和提摩網站的程式結構放在一起比較後，我的結論變得更精確：平台功能更多，不等於現在就應該把所有東西搬過去。

Vercel 的價值，是把 Next.js 網站部署這件事做到非常順；Zeabur、Railway、Render 的價值，是讓網站、API、背景工作與資料服務可以在同一個專案內協作。Supabase 則是獨立的後端服務，並不綁定 Vercel，幾乎可以和這些平台中的任何一個搭配。

真正的一勞永逸，不是選到永遠不用換的平台，而是讓程式保持可攜。

給提摩網站的直接建議

短期保留 Vercel + Supabase，因為目前核心仍是 Next.js 品牌網站與內容後台。當第一個必須常駐的服務出現，例如 n8n、LINE Bot、佇列 worker 或排程，再把該服務放到 Railway、Render 或 Zeabur。只有當多服務管理真的形成日常負擔時，才需要把前端一起集中。

2026 年需要重新認識 Zeabur

Zeabur 已停止讓新專案建立在共享叢集上，新專案改以購買或掛載專屬伺服器為主。因此它現在更像「簡化伺服器與多服務管理的控制台」，而不是單純的低門檻共享 PaaS。這使它更適合已有多個服務、需要固定資源與固定月費的人；只有一個小型網站時，未必是最輕的起點。

其實不是八選一，而是四種思路

模型	典型組合	最適合	主要代價
前端專門化	Vercel / Netlify + Supabase	品牌網站、內容站、Next.js 產品前台	背景服務通常要放到別處
全棧 PaaS	Railway / Render	網站、API、worker、排程、資料庫一起成長	常駐服務按資源計費
伺服器控制台	Zeabur + 專屬或自有伺服器	多個自架服務、固定容量、希望少碰 DevOps	仍要承擔伺服器容量與備援決策
邊緣 / 基礎設施	Cloudflare / Fly.io / VPS	全球低延遲、特殊執行環境、最高控制權	相容性或維運複雜度較高

Supabase 在這張圖裡扮演什麼角色？

Supabase 提供 Postgres、Auth、Storage、Realtime 等後端能力。它和部署 Next.js 的平台是兩個維度：網站可以放在 Vercel、Railway 或 Zeabur，仍然連到同一個 Supabase 專案。這種分工不是架構零散，而是常見的受管服務組合。

選擇原則

先選能最自然承載「目前主要工作負載」的平台，再為真正出現的新服務加第二個平台。不要為了尚未存在的需求，提前承擔整套伺服器維運。

現在的系統其實相當可搬

依目前程式庫檢視，網站使用 Next.js 16、React 19、Supabase SSR / JavaScript SDK，以及標準的 **next build**、**next start** 流程。這代表 Railway、Render、Zeabur、Fly.io 或 VPS 都能以一般 Node.js 服務承載，不需要重寫整個網站。

有利於移轉的地方

- Supabase 已經是外部後端，不需要跟著搬資料庫。
- 沒有發現必須依賴 Vercel 設定檔的部署邏輯。
- 圖片已設為不使用 Next.js 平台影像最佳化，跨平台阻力較低。
- 程式可用標準 Node.js 啟動，不必先容器化。

需要處理的地方

- Vercel Analytics 在其他平台上不一定提供相同資料，應改用其他分析工具或移除。
- 部分圖片與影片仍使用公開的 Vercel Blob 網址；網址通常可繼續讀取，但形成供應商依賴。
- Supabase Auth 的 Site URL 與 Redirect URL 要加入新測試網址和正式網域。
- Server Actions、middleware、ISR / revalidate 與後台登入需在目標平台完整驗證。

移轉難度判斷

移到 Zeabur、Railway 或 Render：低到中等，主要是部署設定與驗證。移到 Cloudflare Workers：中等，因為要透過 OpenNext 並檢查 runtime 相容性。移到 Fly.io 或 VPS：中等，程式改動未必多，但容器、監控、更新與備援工作增加。

部署平台選型矩陣

評分是針對提摩目前的 Next.js + Supabase 情境，5 分代表越有利；不是平台的絕對排名。

平台	Next.js 順手度	多服務 能力	低維運	可攜與 控制	成本 直覺	最適合的起點
Vercel	5	2	5	2	3	Next.js 官網與前台
Zeabur	4	5	4	4	4	固定伺服器上的多服務
Railway	4	5	4	3	3	小型全棧產品與 worker
Render	4	5	4	3	4	傳統 web / worker / cron
Cloudflare	4	4	4	2	5	全球靜態與邊緣應用
Netlify	4	2	5	2	3	內容站與前端團隊
Fly.io	3	5	2	5	3	多區域容器與特殊服務
VPS + Docker	3	5	1	5	5	固定負載與最高控制

怎麼讀這張表

- 現在只想把網站穩定上線：優先看 Vercel。
- 確定近期會新增 API、worker 或排程：優先比較 Railway 與 Render。
- 已有多個服務，願意買一台固定伺服器並希望由平台代管部署：看 Zeabur。
- 流量全球化、靜態內容多、對頻寬成本敏感：評估 Cloudflare。
- 有容器、多區域、網路或執行環境特殊需求：再考慮 Fly.io 或 VPS。

Vercel

Next.js 的原生主場，適合把前端與網站交付做得最省心。

一句話判斷

對提摩目前狀態仍是最自然的預設選擇；Supabase 完全可以繼續搭配，不需要因為未來可能有更多服務就提前搬家。

優勢

- Next.js 功能、預覽部署、CDN、Functions 與框架更新整合最直接。
- 每次提交都能產生預覽網址，適合設計確認與內容審稿。
- 不需要維護常駐伺服器，流量低時通常很輕。
- 和 Supabase、第三方 API、外部儲存服務的串接沒有障礙。

限制

- 常駐程序、長時間 worker、自架 n8n、特殊 TCP 服務不是它的核心場景。
- 使用 Vercel Analytics、Blob、Image Optimization 越多，移轉時要處理的專屬功能越多。
- 使用量計費包含運算、請求與流量等維度，流量放大後要看帳單細項。
- 若前後端服務很多，控制台會分散到不同供應商。

適合

Next.js 品牌官網、作品集、內容網站、行銷頁、SaaS 前台、需要大量預覽部署的設計與產品團隊。

不適合

需要一直執行的 Bot、worker、排程處理、媒體轉檔、自架資料庫，或希望所有容器都在同一個私有網路。

成本型態

有免費 Hobby 層級；付費與超額成本主要依方案和實際資源使用量。適合先小規模開始，但正式商業網站仍應設定用量通知並理解 Functions 與流量計費。

Zeabur

用簡單控制台管理一台專屬或自有伺服器上的多個服務。

一句話判斷

適合確定要跑網站、n8n、Bot、Redis、資料庫等多個常駐服務，並希望以固定伺服器容量換取成本可預測性的人；對單一小網站則可能配置過重。

優勢

- 可在同一專案放前端、後端、資料庫、訊息佇列與物件儲存。
- 模板生態適合快速啟動 n8n、WordPress、Ghost、Postgres、Redis 等服務。
- 能購買 Zeabur 提供的伺服器，或掛載自己在其他雲端購買的機器。
- 伺服器採固定月費，資源獨享，較容易預估穩定負載成本。

限制

- 2026 年共享叢集已停止新建；新專案要先有一台伺服器。
- 控制台簡化部署，但 CPU、RAM、磁碟、備份與單機故障風險仍然存在。
- 只跑一個低流量網站時，固定伺服器可能比按請求計費更浪費。
- 方案費與伺服器費是不同概念；進階備份、監控與支援可能需要付費方案。

適合

個人或小團隊有多個常駐服務，希望少寫 DevOps 設定；需要台灣或亞洲友善操作體驗；可接受固定伺服器並願意規劃備份。

不適合

只有一個幾乎沒有流量的官網、完全不想碰伺服器容量概念，或需要成熟的跨區高可用與企業級 SLA。

成本型態

控制台方案有 Free、Dev、Pro 等層級；服務本身運行在另行購買或自備的伺服器。官方列出的伺服器價格依供應商、地區與規格不同，應把伺服器、備份與方案費合併估算。

Railway

開發者友善的全棧 PaaS，從 GitHub 程式庫快速長出 web、worker、資料庫與私有服務。

一句話判斷

如果提摩近期真的要新增 LINE Bot、API、排程或 worker，Railway 是很自然的第二平台，也比先購買一台專屬伺服器更符合小規模按需成長。

優勢

- Git push 自動部署，支援一般 Node.js 程式與 Docker。
- 同一專案可建立多個服務、變數、資料庫與私有網路。
- 適合 web、API、worker、Bot、cron 與 webhook 等常駐或事件型工作。
- Hobby 方案月費可抵用同額資源，入門成本邏輯相對清楚。

限制

- CPU、記憶體、流量與 volume 都會累積用量，長期常駐服務需要看資源。
- 相較 Vercel，Next.js 的框架專屬功能與預覽體驗沒有那麼原生。
- 低記憶體服務若沒有設定限制，成本可能比預期高。
- 團隊協作與更高資源上限需要較高方案。

適合

小型 SaaS、API、聊天機器人、背景工作、排程、自動化服務，以及想從單一程式庫逐步拆出多個服務的開發者。

不適合

只有靜態網站、追求最完整 Next.js 平台特性，或不願監看任何資源用量。

成本型態

Free 有少量月度額度；Hobby 為每月 5 美元，包含 5 美元資源用量，超過後按 CPU、RAM、網路與儲存使用量計費。可設定用量與資源限制。

Render

結構清楚、偏傳統應用服務的 PaaS，web、private service、worker、cron 與資料庫各司其職。

一句話判斷

當需求比 Vercel 更像後端系統，而且希望服務類型與生命週期清楚分開時，Render 是比 Railway 更偏穩健、傳統的選擇。

優勢

- 明確支援 web service、static site、private service、background worker、cron 與 workflow。
- 支援 Git 自動部署、Docker、健康檢查、私有網路與零停機部署。
- 服務類型與責任邊界清楚，適合較傳統的 API / worker 架構。
- 可使用 Render Postgres、Key Value 和 persistent disk。

限制

- 免費 web service 閒置後會休眠，官方不建議用於正式生產。
- 多個常駐服務會各自產生執行個體費用，總價可能比單機整合高。
- Next.js 可部署，但框架體驗仍不如 Vercel 原生。
- 磁碟、資料庫與跨服務拓樸仍要自行規劃。

適合

Node、Python、Ruby、Go 等後端服務；需要 worker、cron、私有 API、健康檢查、部署前 migration 或明確服務分工的團隊。

不適合

只想部署一個輕量 Next.js 官網，或希望所有服務塞在同一台便宜伺服器共享資源。

成本型態

靜態網站與測試型免費服務可低成本開始；正式 web、worker、private service、資料庫和 cron 依個別執行個體或執行時間計費。

Cloudflare Workers 與 Netlify

兩者都能部署現代網站，但設計哲學不同：Cloudflare 偏全球邊緣運算；Netlify 偏前端工作流程與內容網站。

Cloudflare Workers

優勢

- 靜態資產全球傳遞，Workers 在邊緣執行，延遲與流量成本有吸引力。
- Next.js 可透過 OpenNext 部署，多數 App Router、SSR、ISR、Server Actions 功能已支援。
- 可搭配 D1、KV、R2、Queues、Durable Objects 等邊緣服務。

限制

- 不是標準長駐 Node.js 伺服器，部分 Node API 與套件需要相容性檢查。
- Next.js 需額外 adapter 與建置設定，框架更新時要同步留意相容矩陣。
- 不適合直接搬入任意 Docker、傳統 daemon 或長時間常駐程序。

Netlify

優勢

- Git 工作流程、Deploy Preview、表單、Functions 與靜態站體驗成熟。
- 透過 OpenNext adapter 支援主要 Next.js 功能。
- 適合內容網站、前端團隊與多框架靜態 / serverless 專案。

限制

- 和 Vercel 類似，並非部署資料庫、任意容器與常駐 worker 的全能平台。
- 對 Next.js 的整合雖成熟，仍隔著 adapter，不等同 Vercel 原生。
- 大型網站需理解頻寬、運算與建置用量的計費方式。

對提摩的判斷

Cloudflare 值得在未來流量、全球速度或頻寬成本成為主題時評估；Netlify 則沒有明顯理由取代目前的 Vercel。兩者都不是為了「能跑 n8n 與 Bot」而優先選擇。

Fly.io 與自管 VPS

這兩種方式給你更多控制，也把更多可靠性責任交還給你。

Fly.io

優勢

- 以輕量虛擬機與容器為核心，可把服務部署到多個地區。
- 適合 WebSocket、特殊 runtime、長駐程序、私有網路與靠近使用者的服務。
- CPU、RAM、volume、IP 與地區可細緻配置。

限制

- 需要理解 Machines、區域、volume、網路與自動停止等基礎設施概念。
- volume 是單一地區、單一主機的本機磁碟，高可用與複寫要另行設計。
- 費用由運算、儲存、IP 與流量累積，排錯門檻較高。

自管 VPS + Docker / Coolify

優勢

- 固定月費通常可以在同一台機器跑多個低流量服務，資源利用率高。
- 擁有 root 權限、Docker、網路、檔案系統與供應商選擇權。
- 使用標準容器時，可攜性與長期控制權最高。

限制

- 系統更新、防火牆、SSL、監控、備份、還原、容量與資安都要有人負責。
- 單機故障就是整組服務故障；真正高可用會顯著增加成本與複雜度。
- 省下的平台費，可能換成更多人的維運時間。

對提摩的判斷

目前沒有需要多區容器或自行維運主機的證據。除非把部署管理本身當成核心能力培養，否則 Fly.io 與純 VPS 都不是第一順位。Zeabur 掛載自有 VPS，則是介於全自管與 PaaS 之間的折衷。

提摩可以怎麼組，而不是只問放哪裡

方案	組合	適用時機	判斷
A · 維持現況	Vercel + Supabase	官網、作品集、詢價、CMS 為主	現在最合理
B · 混合式	Vercel + Supabase + Railway / Render 服務	新增 Bot、worker、webhook、排程，但前端仍穩定	最推薦的成長路徑
C · 集中 PaaS	Railway / Render + Supabase	前端與多個程式服務需要統一管理	等管理成本出現再搬
D · 固定伺服器	Zeabur + 專屬伺服器 + Supabase	同時跑多個常駐服務，資源需求穩定	有規模後很合適
E · 全自管	VPS / Fly.io + Docker + Supabase	有特殊 runtime、網路、多區或控制要求	目前不需要

為什麼混合式不是壞事？

把最適合 Next.js 的前端留在 Vercel，把真正需要常駐的服務放到 Railway 或 Render，再讓兩者共同連接 Supabase，是清楚的責任分工。只有當跨平台的帳號、監控、環境變數與費用管理開始造成負擔時，集中化才會產生明顯價值。

避免供應商綁定的五件事

使用標準環境變數；維持 next build / next start；把上傳檔案集中到可替換的物件儲存；分析工具不要成為核心流程；需要常駐服務時補一份 Dockerfile。做到這些，比現在選哪一個品牌更接近真正的一勞永逸。

沒有正式網域，正是最低風險的測試期

因為提摩網站還沒有正式網域，可以讓 Vercel 與候選平台同時存在，使用各自的測試網址連到同一個 Supabase。這不是一次性切換，而是一場可回復的平行驗證。

步驟	要做什麼	驗證重點
1 · 建立測試部署	從 GitHub 匯入；設定 <code>pnpm build / pnpm start</code> 或平台自動偵測	建置成功、PORT 綁定、Node 版本
2 · 複製環境變數	加入 Supabase URL、anon key 與其他非公開密鑰	不要把正式 secrets 寫進程式庫
3 · 更新 Auth 網址	把候選平台測試網址加入 Supabase redirect allowlist	後台登入、登出、cookie、middleware
4 · 功能回歸	測試首頁、作品、聯絡、詢價、Studio、圖片與影片	Server Actions、revalidate、上傳與讀取
5 · 觀察數日	比較啟動延遲、錯誤、log、資源與帳單	不要只看第一次開站是否成功
6 · 綁定正式網域	先設定 DNS，再更新 Auth Site URL 與 SEO canonical	HTTPS、www / apex、重新導向
7 · 保留回復路徑	Vercel 暫留一段觀察期，確認穩定後再下線	DNS TTL、舊網址、媒體與分析資料

最容易被忽略的風險

Vercel Blob 公開網址不會因為網站搬家就立刻失效，但它仍是外部依賴。正式切換前應列出所有媒體來源，決定哪些保留、哪些移到 Supabase Storage 或專案 public 目錄，避免之後刪除 Vercel 專案時才發現素材一起消失。

提摩網站的分階段選擇

現在

維持 Vercel + Supabase。這是對目前需求最小、最穩、最容易維護的配置。先把正式網域、內容、詢價流程、後台與分析做好。

第一個常駐服務出現時

優先在 Railway 或 Render 部署該服務，前端暫時不搬。Railway 適合快速試做與小型全棧服務；Render 適合服務分工、worker、cron 和私有網路較明確的架構。

服務增加到值得買固定伺服器時

再評估 Zeabur。當 n8n、LINE Bot、Redis、後台 API、排程等服務會長期運行，而且總資源可預估時，把它們集中到一台由 Zeabur 管理的伺服器，固定成本與操作便利才會真正發揮。

效能或控制成為核心需求時

才考慮 Cloudflare Workers、Fly.io 或自管 VPS。這些選擇的價值在全球邊緣、特殊 runtime、多區部署與基礎設施控制，不是單純為了把一般官網上線。

最後的心得

我現在不會把 Zeabur 視為一開始就必須採用的終點，而會把它視為系統進入「多個常駐服務」階段時很有吸引力的集中方案。對現在的提摩而言，Vercel 不是暫時將就，而是符合主要工作負載的選擇；Railway / Render 是下一階段的自然延伸；Zeabur 則適合服務規模足以合理使用一台固定伺服器之後。

好架構不是永遠不搬家，而是每一次搬家都不需要重寫。

官方文件與查核日期

以下資料均於 2026 年 8 月 3 日查核。價格與方案可能調整，實際採用前應再次查看官方頁面。

來源	用途
Vercel Pricing	計費模型與受管基礎設施
Vercel Functions	Functions 執行與適用場景
Zeabur Platform Overview	平台定位、多服務與共享叢集狀態
Zeabur Shared Cluster (Deprecated)	新專案停止使用共享叢集
Zeabur Server	購買或掛載伺服器、固定月費
Zeabur Pricing	Free、Dev、Pro 與方案能力
Railway Pricing Plans	方案、額度與資源用量計費
Railway Cost Control	用量與資源限制
Render Service Types	web、worker、cron、private service 與 datastore
Render Deploys	自動部署、zero downtime 與 ephemeral filesystem
Render Free	免費服務限制與正式環境建議
Cloudflare Workers Pricing	Free / Paid 與請求、CPU 計費
Next.js on Cloudflare Workers	OpenNext 與 Next.js 功能相容性
Next.js on Netlify	OpenNext adapter 與主要功能
Fly.io Pricing	Machines、volume、網路與按用量計費
Fly Volumes	本機 volume、複寫與高可用限制
DigitalOcean App Platform Pricing	傳統受管 PaaS 的容器與元件計價參考

專案現況依據

提摩網站本機程式庫檢視：package.json、next.config.mjs、Supabase client / server / middleware、Vercel Analytics 與公開媒體網址。檢視日期同上。

本文件是技術選型心得與規劃參考，不構成任何平台的價格保證或 SLA 承諾。